1. **Design, Develop and Implement a menu-driven Program for list data structure in C/C++ for the following operations.**

a. Creating an Empty list of N Char Items
b. Inserting an Item at a given valid Position (i)
c. Deleting an Item at a given valid Position (i)
d. Searching an Item and show its Position in List.
e. Finding total number of elements.
f. Print all elements.
g. Exit.

Support the program with functions for each of the above operations. Discuss their best case, worst case, and average case time complexity

<u>Answer:</u>

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class LinkedList {
private:
    Node* head = nullptr;

public:
    void createList() {
        cout << "Enter five integer elements: " << endl;
        for (int i = 0; i < 5; i++) {
            int element;
            cin >> element;
            insertAtEnd(element);
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node {data, nullptr};
        if (!head)
            head = newNode;
        else {
            Node* current = head;
            while (current->next)
                current = current->next;
            current->next = newNode;
        }
```

```
    cout << "Inserted " << data << " at the end." << endl;
}

void printList() {
    if (!head) {
        cout << "Empty list." << endl;
        return;
    }
    Node* current = head;
    cout << "Elements: ";
    while (current) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}

void insertAtPosition(int data, int position) {
    Node* newNode = new Node {data, nullptr};
    if (position == 1) {
        newNode->next = head;
        head = newNode;
    } else {
        Node* current = head;
        Node* previous = nullptr;
        int count = 1;
        while (count < position && current) {
            previous = current;
            current = current->next;
            count++;
        }
        if (!current && count == position)
            previous->next = newNode;
        else if (current) {
            newNode->next = current;
            previous->next = newNode;
        } else
            cout << "Invalid position." << endl;
    }
    cout << "Inserted " << data << " at position " << position << "." << endl;
}

void deleteAtPosition(int position) {
    if (!head) {
```

```
            cout << "Empty list." << endl;
            return;
        }
        Node* current = head;
        Node* previous = nullptr;
        int count = 1;
        while (count < position && current) {
            previous = current;
            current = current->next;
            count++;
        }
        if (!current)
            cout << "Invalid position." << endl;
        else if (!previous) {
            head = current->next;
            delete current;
        } else {
            previous->next = current->next;
            delete current;
        }
        cout << "Deleted from position " << position << "." << endl;
    }
};

int main() {
    LinkedList linkedList;
    int choice, element, position;
    do {
        cout << "Linked List Menu" << endl
             << "1. Create list" << endl
             << "2. Print list" << endl
             << "3. Insert element at position" << endl
             << "4. Delete element at position" << endl
             << "5. Quit" << endl
             << "Choice: ";
        cin >> choice;
        switch (choice) {
            case 1: linkedList.createList(); break;
            case 2: linkedList.printList(); break;
            case 3: cout << "Element, position: "; cin >> element >> position;
                    linkedList.insertAtPosition(element, position); break;
            case 4: cout << "Position: "; cin >> position;
                    linkedList.deleteAtPosition(position); break;
            case 5: cout << "Exiting." << endl; break;
```

```
            default: cout << "Invalid choice." << endl; break;
        }
    } while (choice != 5);
    return 0;
}
```

Output-

```
Enter the number of items in the array: 3
Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 1
Enter the item to be inserted: 0
Enter the position: 0

Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 1
Enter the item to be inserted: 1
Enter the position: 1
```

```
Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 1
Enter the item to be inserted: 2
Enter the position: 2

Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 5
Array elements: 0 1 2

Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 4
Total number of elements: 3
```

```
Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 2
Enter the position of the item to be deleted: 1

Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 5
Array elements: 0 2

Menu
1. Insert an item
2. Delete an item
3. Search for an item
4. Get the total number of elements
5. Print all elements
6. Exit
Enter your choice: 6
Exiting the program.
```

2. **Design, Develop and Implement an algorithm in C/C++ for the following sorting techniques.**

a. Selection Sort b. Bubble Sort c. Insertion Sort

Discuss their best case, worst case and average case time complexity.

Answer:

```
#include <iostream>
using namespace std;
void InsertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && key < a[j]) {
            swap(a[j], a[j + 1]);
            j--;
        }
    }
}
```

Best Case	**O**(n)
Average Case	O(n^2)
Worst Case	O(n^2)

```
void InsertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && key < a[j]) {
            swap(a[j], a[j + 1]);
            j--;
        }
    }
}
```

Best Case	**O**(n^2)
Average Case	O(n^2)
Worst Case	O(n^2)

```
void InsertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && key < a[j]) {
            swap(a[j], a[j + 1]);
            j--;
        }
    }
}
```

Best Case	**O(n)**
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

```
void printArray(int array[], int size) {
    for (int i = 0; i < size; i++)
        cout << array[i] << " ";
    cout << endl;
}

int main() {
    int n;
    int size;
    cout << "Enter size of array\n";
    cin >> size;
    int a[size];
    cout << "Enter array elements\n";
    for (int i = 0; i < size; i++)
        cin >> a[i];
    cout << "Enter 1 for Selection sort, 2 for Bubble sort, 3 for Insertion sort\n";
    cin >> n;
    switch (n) {
        case 1:
            SelectionSort(a, size);
            printArray(a, size);
            break;
        case 2:
            bubbleSort(a, size);
            printArray(a, size);
            break;
```

```
 case 3:
        InsertionSort(a, size);
        printArray(a, size);
        break;
   }
   return 0;
}
```

Output-

```
Enter size of array
5
Enter array elements
3
2
1
5
4
Enter 1 for Selection sort, 2 for Bubble sort, 3 for Insertion sort
1
1 2 3 4 5
```

3. Design, Develop and Implement a menu-driven Program in C/C++ for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)

a. Create a stack.
b. Push an Element onto Stack
c. Pop an Element from Stack and Print it.
d. Demonstrate Overflow and Underflow situations on Stack
e. Display the top element of the Stack.
f. Exit

Support the program with appropriate functions for each of the above operations

Answer:

```
#include<iostream>
using namespace std;
int mystack[5];
int top = -1;
void push() {
   int x;
   cin >> x;
   if (top == 4) {
      cout << "Overflow" << endl;
      return;
   }
   else {
      top++;
      mystack[top] = x;
   }
}
void pop() {
   if (top == -1) {
      cout << "Underflow" << endl;
      return;
   }
   else {
      top--;
   }
}
void peak() {
   if (top == -1) {
      cout << "Empty stack" << endl;
      return;
   }
   cout << mystack[top] << endl;
}
```

```
void display() {
    for (int i = top; i >= 0; i--) {
        cout << mystack[i] << " ";
    }
    cout << endl;
}
int main() {
    int n;
    int operations;
    cin >> operations;
   for (int i = 0; i < operations; i++) {
        cin >> n;
        switch (n) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                peak();
                break;
            case 4:
                return 0;
        }
        display();
    }
    return 0;
}
```

Output-

```
4
1
56
56
1
85
85 56
2
56
3
56
56
```

4. Design, Develop and Implement a Program in C for the following Stack Applications
a. Converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.
b. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^
c. Solving Tower of Hanoi problem with n disks

<u>Answer:</u>

a)

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;
int performOperation(char op, int operand1, int operand2) {
   switch (op) {
      case '+':
         return operand1 + operand2;
      case '-':
         return operand1 - operand2;
      case '*':
         return operand1 * operand2;
      case '/':
         return operand1 / operand2;
      case '%':
         return operand1 % operand2;
      case '^': {
         int result = 1;
         for (int i = 0; i < operand2; i++) {
            result *= operand1;
         }
         return result;
      }
      default:
         return 0; // Invalid operator
   }
}
int evaluatePostfix(const string& expression) {
   stack<int> stk;

   for (char token : expression) {
      if (isdigit(token)) {
         stk.push(token - '0'); // Convert char to integer
      }
```

```
        else if (token == '+' || token == '-' || token == '*' || token == '/' || token == '%' || token == '^')
{
            int operand2 = stk.top();
            stk.pop();
            int operand1 = stk.top();
            stk.pop();
            int result = performOperation(token, operand1, operand2);
            stk.push(result);
        }
    }
return stk.top();
}
int main() {
    string postfixExpression;
    cout << "Enter a postfix expression: ";
    getline(cin, postfixExpression);
    int result = evaluatePostfix(postfixExpression);
    cout << "Evaluation result: " << result << endl;
    return 0;
}
```

Output-

```
Enter a postfix expression: 5 6 2 + * 12 - 4 / -
Evaluation result: 40
```

b)

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int precedence(char op) {
    if (op == '^')
        return 3;
    else if (op == '*' || op == '/' || op == '%')
        return 2;
    else if (op == '+' || op == '-')
        return 1;
    else
        return -1; // Invalid operator
}

bool isOperand(char token) {
    return isalnum(token);
```

```
}

bool isOperator(char token) {
    return (token == '+' || token == '-' || token == '*' || token == '/' || token == '%' || token == '^');
}

string infixToPostfix(const string& expression) {
    stack<char> stk;
    string postfix;

    for (char token : expression) {
        if (token == ' ' || token == '\t')
            continue;

        if (isOperand(token)) {
            postfix += token;
        }
        else if (isOperator(token)) {
            while (!stk.empty() && stk.top() != '(' && precedence(stk.top()) >= precedence(token))
{
                postfix += stk.top();
                stk.pop();
            }
            stk.push(token);
        }
        else if (token == '(') {
            stk.push(token);
        }
        else if (token == ')') {
            while (!stk.empty() && stk.top() != '(') {
                postfix += stk.top();
                stk.pop();
            }
            if (!stk.empty() && stk.top() == '(')
                stk.pop();
        }
    }

    while (!stk.empty()) {
        postfix += stk.top();
        stk.pop();
    }

    return postfix;
```

```
}

int main() {
    string infixExpression;
    cout << "Enter an infix expression: ";
    getline(cin, infixExpression);

    string postfixExpression = infixToPostfix(infixExpression);
    cout << "Postfix expression: " << postfixExpression << endl;
    return 0;
}
```

Output-

```
Enter an infix expression: (A + B / C * ( D + E ) - F )
Postfix expression: ABC/DE+*+F-
```

c)

```
#include <iostream>
using namespace std;

void towerofHanoi(int n, char source, char auxillary, char destination) {
    if (n == 0) {
        return;
    }
    towerofHanoi(n - 1, source, destination, auxillary);
    cout << source << " " << destination << endl;
    towerofHanoi(n - 1, auxillary, source, destination);
}
int main() {
    int n;
    cin >> n;
    towerofHanoi(n, 'a', 'b', 'c');
    return 0;
}
```

Output-

```
3
Move a to c
Move a to b
Move c to b
Move a to c
Move b to a
Move b to c
Move a to c
```

5. **Write a program that implements Queue (its operations) using i) Arrays ii) Linked list (Pointers).**

Answer:

```
#include<iostream>
using namespace std;
int myQueue[5];
int front = -1;
int rear = -1;
void enqueue() {
   int data;
   cin >> data;
   if (rear == 4) {
      cout << "Overflow" << endl;
   }
   else if (front == -1 && rear == -1) {
      front = rear = 0;
      myQueue[rear] = data;
   }
   else {
      rear++;
      myQueue[rear] = data;}
}
void dequeue() {
   if (rear == -1 && front == -1) {
      cout << "Underflow" << endl;
   }
   else if (front == rear) {
      front = rear = -1;
   }
   else {
      front++;
   }
}
void display() {
   if (front == -1 && rear == -1) {
      cout << "Queue is empty" << endl;
   }
   else {
      for (int i = front; i < rear + 1; i++) {
         cout << myQueue[i] << " ";
      }
      cout << endl;
   }
}
```

```
void peek() {
    if (front == -1 && rear == -1) {
        cout << "Queue is empty" << endl;
    }
    else {
        cout << myQueue[front] << endl;
    }
}
int main() {
    int n;
    int operations;
    cin >> operations;
    // NUMBER OF OPERATIONS
    for (int i = 0; i < operations; i++) {
        cin >> n;
        switch (n) {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
                break;
            case 3:
                peek();
                break;
        }
        display();
    }
    return 0;
}
```

Output-

```
6
1
45
45
1
56
45  56
1
78
45  56  78
2
56  78
3
56
56  78
2
78
```

b)-

```
#include <iostream>
using namespace std;
struct Node {
   int data;
   struct Node* next;
};
struct Node* front = NULL;
struct Node* rear = NULL;
struct Node* temp;
void enqueue() {
   int val;
   cin >> val;
   if (rear == NULL) {
      rear = new Node;
      rear->next = NULL;
      rear->data = val;
      front = rear;
   } else {
      temp = new Node;
      rear->next = temp;
      temp->data = val;
      temp->next = NULL;
      rear = temp;}
}
void dequeue() {
   temp = front;
   if (front == NULL) {
      cout << "Underflow" << endl;
      return;
   } else if (temp->next != NULL) {
      temp = temp->next;
      delete front;
      front = temp;
   } else {
      delete front;
      front = NULL;
      rear = NULL;}
}
void display() {
   temp = front;
   if (front == NULL && rear == NULL) {
      cout << "Queue is empty" << endl;
      return;
```

```
    }
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
void peak() {
    cout << front->data << endl;
}
int main() {
    int operations;
    cin >> operations;
    int n;
    for (int i = 0; i < operations; i++) {
        cin >> n;
        switch (n) {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
                break;
            case 3:
                peak();
                break;
        }
        display();
    }
    return 0;
}
```

Output-

```
5
1
45
45
156
45
1
56
45  56
2
56
3
56
56
```

6. Write a program that implement Circular Queue (its operations) using Arrays.

Answer:

```
#include <iostream>
using namespace std;

const int MAX_SIZE = 5;

class CircularQueue {
private:
    int front, rear;
    int queue[MAX_SIZE];

public:
    CircularQueue() {
        front = rear = -1;
    }

    bool isEmpty() {
        return (front == -1 && rear == -1);
    }

    bool isFull() {
        return ((rear + 1) % MAX_SIZE == front);
    }

    void enqueue(int item) {
        if (isFull()) {
            cout << "Queue is full. Unable to enqueue.\n";
            return;
        } else if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % MAX_SIZE;
        }

        queue[rear] = item;
        cout << "Enqueued: " << item << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty. Unable to dequeue.\n";
            return;
        } else if (front == rear) {
```

```
            cout << "Dequeued: " << queue[front] << endl;
            front = rear = -1;
        } else {
            cout << "Dequeued: " << queue[front] << endl;
            front = (front + 1) % MAX_SIZE;
        }
    }

    void displayQueue() {
        if (isEmpty()) {
            cout << "Queue is empty.\n";
            return;
        }

        int i = front;
        cout << "Queue elements: ";
        while (i != rear) {
            cout << queue[i] << " ";
            i = (i + 1) % MAX_SIZE;
        }
        cout << queue[i] << endl;
    }
};

int main() {
    CircularQueue cq;
    int choice;
    int item;

    do {
        cout << "Menu\n";
        cout << "1. Enqueue\n";
        cout << "2. Dequeue\n";
        cout << "3. Display Queue\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter the element to enqueue: ";
                cin >> item;
                cq.enqueue(item);
                break;
```

```
            case 2:
                cq.dequeue();
                break;
            case 3:
                cq.displayQueue();
                break;
            case 4:
                cout << "Exiting the program.\n";
                break;
            default:
                cout << "Invalid choice. Please try again.\n";
                break;
        }

        cout << endl;
    } while (choice != 4);

    return 0;
}
```

Output-

```
5
1
45
45
1
85
45  85
1
47
45  85  47
2
85  47
3
85
85  47
```

7. **Design, Develop and Implement a menu driven Program in C for priority queue with enqueue and dequeue operation using array.**

Answer:

```
#include <iostream>
using namespace std;
const int MAX_SIZE = 10;
struct QueueElement {
   int data;
   int priority;};
class PriorityQueue {
private:
   QueueElement elements[MAX_SIZE];
   int rear;
public:
   PriorityQueue() {
      rear = -1;}
   void enqueue(int data, int priority) {
      if (rear == MAX_SIZE - 1) {
         cout << "Queue is full. Cannot enqueue element." << endl;
         return;}
      int i = rear;
      while (i >= 0 && elements[i].priority > priority) {
         elements[i + 1] = elements[i];
         i--;}
      elements[i + 1].data = data;
      elements[i + 1].priority = priority;
      rear++;
      cout << "Enqueued element " << data << " with priority " << priority << endl;}
   void dequeue() {
      if (rear == -1) {
         cout << "Queue is empty. Cannot dequeue element." << endl;
         return;}
      QueueElement dequeuedElement = elements[rear];
      rear--;
      cout << "Dequeued element " << dequeuedElement.data << " with priority " <<
dequeuedElement.priority << endl;};
int main() {
   PriorityQueue queue;
   int choice, data, priority;
   do {
      cout << "Priority Queue Menu" << endl;
      cout << "1. Enqueue" << endl;
      cout << "2. Dequeue" << endl;
      cout << "3. Quit" << endl;
```

```
    cout << "Enter your choice: ";
    cin >> choice;
    switch (choice) {
        case 1:  cout << "Enter the element to enqueue: ";
            cin >> data;
            cout << "Enter the priority: ";
            cin >> priority;
            queue.enqueue(data, priority);
            break;
        case 2: queue.dequeue();
            break;
        case 3: cout << "Exiting the program..." << endl;
            break;
        default:cout << "Invalid choice. Please try again." << endl;
            break;}
    cout << endl;
  } while (choice != 3);
  return                                                    0;}
```

```
Priority Queue Menu
1. Enqueue
2. Dequeue
3. Quit
Enter your choice: 1
Enter the element to enqueue: 56
Enter the priority: 2
Enqueued element 56 with priority 2

Priority Queue Menu
1. Enqueue
2. Dequeue
3. Quit
Enter your choice: 1
Enter the element to enqueue: 65
Enter the priority: 8
Enqueued element 65 with priority 8

Priority Queue Menu
1. Enqueue
2. Dequeue
3. Quit
Enter your choice: 2
Dequeued element 65 with priority 8
```

8. Design, Develop and Implement a menu driven Program in C for singly link list using structure with following functionality.

1. create a link list of five user given integer elements.
2. create a function to print all the elements of the link list.
3. create a function to insert an integer element at any user given position.
4. create a function to delete an element at any user given position

Answer:

```
#include <iostream>
using namespace std;
struct Node {
   int data;
   Node* next;
};
class LinkedList {
private:
   Node* head = nullptr;
public:
   void createList() {
      cout << "Enter five integer elements: " << endl;
      for (int i = 0; i < 5; i++) {
         int element;
         cin >> element;
         insertAtEnd(element);
      }
   }
   void insertAtEnd(int data) {
      Node* newNode = new Node {data, nullptr};
      if (!head)
         head = newNode;
      else {
         Node* current = head;
         while (current->next)
            current = current->next;
         current->next = newNode;
      }
      cout << "Inserted " << data << " at the end." << endl;
   }
   void printList() {
      if (!head) {
         cout << "Empty list." << endl;
         return;
      }
```

```
    Node* current = head;
    cout << "Elements: ";
    while (current) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}
void insertAtPosition(int data, int position) {
    Node* newNode = new Node {data, nullptr};
    if (position == 1) {
        newNode->next = head;
        head = newNode;
    } else {
        Node* current = head;
        Node* previous = nullptr;
        int count = 1;
        while (count < position && current) {
            previous = current;
            current = current->next;
            count++;
        }
        if (!current && count == position)
            previous->next = newNode;
        else if (current) {
            newNode->next = current;
            previous->next = newNode;
        } else
            cout << "Invalid position." << endl;
    }
    cout << "Inserted " << data << " at position " << position << "." << endl;
}
void deleteAtPosition(int position) {
    if (!head) {
        cout << "Empty list." << endl;
        return;
    }
    Node* current = head;
    Node* previous = nullptr;
    int count = 1;
    while (count < position && current) {
        previous = current;
        current = current->next;
        count++;
```

```
        }
        if (!current)
            cout << "Invalid position." << endl;
        else if (!previous) {
            head = current->next;
            delete current;
        } else {
            previous->next = current->next;
            delete current;
        }
        cout << "Deleted from position " << position << "." << endl;
    }
};
int main() {
    LinkedList linkedList;
    int choice, element, position;
    do {
        cout << "Linked List Menu" << endl;
        cout << "1. Create list" << endl;
        cout << "2. Print list" << endl;
        cout << "3. Insert element at position" << endl;
        cout << "4. Delete element at position" << endl;
        cout << "5. Quit" << endl;
        cout << "Choice: ";
        cin >> choice;
        switch (choice) {
            case 1: linkedList.createList(); break;
            case 2: linkedList.printList(); break;
            case 3: cout << "Element, position: "; cin >> element >> position;
                    linkedList.insertAtPosition(element, position); break;
            case 4: cout << "Position: "; cin >> position;
                    linkedList.deleteAtPosition(position); break;
            case 5: cout << "Exiting." << endl; break;
            default: cout << "Invalid choice." << endl; break;
        }
    } while (choice != 5);
    return 0;
}
```

Output-

```
Linked List Menu
1. Create a linked list of five user-given integer elements
2. Print all elements of the linked list
3. Insert an element at a user-given position
4. Delete an element at a user-given position
5. Quit
Enter your choice: 1
Enter five integer elements to create the linked list:
5
Inserted element 5 at the end.
4
Inserted element 4 at the end.
89
Inserted element 89 at the end.
65
Inserted element 65 at the end.
36
Inserted element 36 at the end.

Linked List Menu
1. Create a linked list of five user-given integer elements
2. Print all elements of the linked list
3. Insert an element at a user-given position
4. Delete an element at a user-given position
5. Quit
```

```
Enter your choice: 2
Linked list elements: 5 4 89 65 36

Linked List Menu
1. Create a linked list of five user-given integer elements
2. Print all elements of the linked list
3. Insert an element at a user-given position
4. Delete an element at a user-given position
5. Quit
Enter your choice: 3
Enter the element to insert: 78
Enter the position to insert at: 2
Inserted element 78 at position 2.

Linked List Menu
1. Create a linked list of five user-given integer elements
2. Print all elements of the linked list
3. Insert an element at a user-given position
4. Delete an element at a user-given position
5. Quit
Enter your choice: 3
Enter the element to insert: 78
Enter the position to insert at: 2
Inserted element 78 at position 2.
```

9. Design, Develop and Implement a menu driven Program in C for a circular link list using structure with following functionality.

1. create a link list of eight user given character elements.
2. a function to print all the elements of the link list.
3. a function to reverse the lisk list.
4. a function to sort the element of link list in ascending order

Answer:

```
#include <iostream>
using namespace std;
struct Node {
    char data;
    Node* next;
};
class CircularLinkedList {
private:
    Node* head = nullptr;
public:
    void createList() {
        cout << "Enter eight character elements: " << endl;
        for (int i = 0; i < 8; i++) {
            char element;
            cin >> element;
            insertAtEnd(element);
        }
    }
    void insertAtEnd(char data) {
        Node* newNode = new Node {data, nullptr};
        if (!head) {
            head = newNode;
            newNode->next = head;
        } else {
            Node* current = head;
            while (current->next != head)
                current = current->next;
            current->next = newNode;
            newNode->next = head;
        }
        cout << "Inserted " << data << " at the end." << endl;
    }
    void printList() {
        if (!head) {
            cout << "Empty list." << endl;
            return;
```

```
        }
        Node* current = head;
        cout << "Elements: ";
        do {
            cout << current->data << " ";
            current = current->next;
        } while (current != head);
        cout << endl;
    }
    void reverseList() {
        if (!head) {
            cout << "Empty list. Cannot reverse." << endl;
            return;
        }
        Node *prev = nullptr, *next = nullptr, *current = head;
        do {
            next = current->next;
            current->next = prev;
            prev = current;
            current = next;
        } while (current != head);
        head->next = prev;
        head = prev;
        cout << "Reversed successfully." << endl;
    }
    void sortList() {
        if (!head) {
            cout << "Empty list. Cannot sort." << endl;
            return;
        }
        Node* current = head;
        do {
            Node* index = current->next;
            while (index != head) {
                if (current->data > index->data)
                    swap(current->data, index->data);
                index = index->next;
            }
            current = current->next;
        } while (current != head);
        cout << "Sorted ascending." << endl;
    }
};
int main() {
```

```
    CircularLinkedList circularLinkedList;
    int choice;
    do {
        cout << "Circular Linked List Menu" << endl;
        cout << "1. Create list" << endl;
        cout << "2. Print list" << endl;
        cout << "3. Reverse list" << endl;
        cout << "4. Sort list" << endl;
        cout << "5. Quit" << endl;
        cout << "Choice: ";
        cin >> choice;
        switch (choice) {
            case 1: circularLinkedList.createList(); break;
            case 2: circularLinkedList.printList(); break;
            case 3: circularLinkedList.reverseList(); break;
            case 4: circularLinkedList.sortList(); break;
            case 5: cout << "Exiting..." << endl; break;
            default: cout << "Invalid choice." << endl; break;
        }
        cout << endl;
    } while (choice != 5);
    return 0;
}
```

Output-

```
Circular Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: 1
Enter eight character elements to create the linked list:
56
Inserted element 5 at the end.
Inserted element 6 at the end.
6
Inserted element 6 at the end.
64
Inserted element 6 at the end.
Inserted element 4 at the end.
4
Inserted element 4 at the end.
8
Inserted element 8 at the end.
96
Inserted element 9 at the end.
```

10. Implement a doubly linked list using structure with following functionality.

1. create a link list of eight user given character elements.

2. a function to print all the elements of the link list.

3. a function to reverse the link list.

4. a function to sort the elements of a link list in ascending order.

Answer:

```
#include <iostream>
using namespace std;
struct Node {
   char data;
   Node* prev;
   Node* next;
};

class DoublyLinkedList {
private:
   Node* head;
   Node* tail;
public:
   DoublyLinkedList() {
      head = NULL;
      tail = NULL;
   }
   void createList() {
      cout << "Enter eight character elements to create the linked list: " << endl;
      for (int i = 0; i < 8; i++) {
         char element;
         cin >> element;
         insertAtEnd(element);
      }
   }
   void insertAtEnd(char data) {
      Node* newNode = new Node;
      newNode->data = data;
      newNode->prev = NULL;
      newNode->next = NULL;
      if (head == NULL) {
         head = newNode;
         tail = newNode;
      } else {
         tail->next = newNode;
         newNode->prev = tail;
         tail = newNode;
```

```
    }

    cout << "Inserted element " << data << " at the end." << endl;
}
void printList() {
    if (head == NULL) {
        cout << "Linked list is empty." << endl;
        return;
    }
    Node* current = head;
    cout << "Linked list elements: ";
    while (current != NULL) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}
void reverseList() {
    if (head == NULL) {
        cout << "Linked list is empty. Cannot reverse." << endl;
        return;
    }
    Node* current = head;
    Node* temp = NULL;
    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }
    if (temp != NULL) {
        head = temp->prev;
    }

    cout << "Linked list reversed successfully." << endl;
}
void sortList() {
    if (head == NULL) {
        cout << "Linked list is empty. Cannot sort." << endl;
        return;
    }
    Node* current = head;
    Node* index = NULL;
    char temp;
```

```
        while (current != NULL) {
            index = current->next;
            while (index != NULL) {
                if (current->data > index->data) {
                    temp = current->data;
                    current->data = index->data;
                    index->data = temp;
                }
                index = index->next;
            }
            current = current->next;
        }
        cout << "Linked list sorted in ascending order." << endl;
    }
};
int main() {
    DoublyLinkedList doublyLinkedList;
    int choice;
    char element;
    do {
        cout << "Doubly Linked List Menu" << endl;
        cout << "1. Create a linked list of eight user-given character elements" << endl;
        cout << "2. Print all elements of the linked list" << endl;
        cout << "3. Reverse the linked list" << endl;
        cout << "4. Sort the elements of the linked list in ascending order" << endl;
        cout << "5. Quit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                doublyLinkedList.createList();
                break;
            case 2:
                doublyLinkedList.printList();
                break;
            case 3:
                doublyLinkedList.reverseList();
                break;
            case 4:
                doublyLinkedList.sortList();
                break;
            case 5:
                cout << "Exiting the program..." << endl;
                break;
```

```
            default:
                cout << "Invalid choice. Please try again." << endl;
                break;
        }
        cout << endl;
    } while (choice != 5);
    return 0;
}
```

Output-

```
Doubly Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: 1
Enter eight character elements to create the linked list:
4
Inserted element 4 at the end.
5
Inserted element 5 at the end.
7
Inserted element 7 at the end.
8
9Inserted element 8 at the end.

6
6
Inserted element 6 at the end.
12
Inserted element 1 at the end.
Inserted element 2 at the end.
23
Inserted element 2 at the end.
```

```
Doubly Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: Linked list reversed successfully.

Doubly Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: 2
Linked list elements: 2 2 1 6 8 7 5 4

Doubly Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: 3
Linked list reversed successfully.
```

```
Doubly Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: 4
Linked list sorted in ascending order.

Doubly Linked List Menu
1. Create a linked list of eight user-given character elements
2. Print all elements of the linked list
3. Reverse the linked list
4. Sort the elements of the linked list in ascending order
5. Quit
Enter your choice: 5
Exiting the program...
```

11. Design, Develop and Implement a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers

a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2

b. Traverse the BST in Inorder, Preorder and Post Order

c. Search the BST for a given element (KEY) and report the appropriate message

d. Exit

Answer:

```
#include <iostream>
using namespace std;
struct Node {
   int data;
   Node* left;
   Node* right;
};
class BinarySearchTree {
private:
   Node* root;
public:
   BinarySearchTree() {
      root = NULL;
   }
   Node* createNode(int data) {
      Node* newNode = new Node;
      newNode->data = data;
      newNode->left = NULL;
      newNode->right = NULL;
      return newNode;
   }
   Node* insertNode(Node* root, int data) {
      if (root == NULL) {
         root = createNode(data);
      } else if (data <= root->data) {
         root->left = insertNode(root->left, data);
      } else {
         root->right = insertNode(root->right, data);
      }
      return root;
   }
   void createBST() {
      int elements[] = {6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2};
      int size = sizeof(elements) / sizeof(elements[0]);
      for (int i = 0; i < size; i++) {
         root = insertNode(root, elements[i]);
      }
```

```
        cout << "BST created with elements: ";
        for (int i = 0; i < size; i++) {
            cout << elements[i] << " ";
        }
        cout << endl;
    }
    void inorderTraversal(Node* root) {
        if (root != NULL) {
            inorderTraversal(root->left);
            cout << root->data << " ";
            inorderTraversal(root->right);
        }
    }
    void preorderTraversal(Node* root) {
        if (root != NULL) {
            cout << root->data << " ";
            preorderTraversal(root->left);
            preorderTraversal(root->right);
        }
    }
    void postorderTraversal(Node* root) {
        if (root != NULL) {
            postorderTraversal(root->left);
            postorderTraversal(root->right);
            cout << root->data << " ";
        }
    }
    bool searchElement(Node* root, int key) {
        if (root == NULL) {
            return false;
        } else if (root->data == key) {
            return true;
        } else if (key <= root->data) {
            return searchElement(root->left, key);
        } else {
            return searchElement(root->right, key);
        }
    }
};
int main() {
    BinarySearchTree bst;
    int choice, key;
    do {
        cout << "Binary Search Tree (BST) Menu" << endl;
```

```
cout << "1. Create a BST of N Integers" << endl;
cout << "2. Traverse the BST in Inorder" << endl;
cout << "3. Traverse the BST in Preorder" << endl;
cout << "4. Traverse the BST in Postorder" << endl;
cout << "5. Search the BST for a given element" << endl;
cout << "6. Exit" << endl;
cout << "Enter your choice: ";
cin >> choice;
switch (choice) {
    case 1:
        bst.createBST();
        break;
    case 2:
        cout << "Inorder Traversal: ";
        bst.inorderTraversal(bst.root);
        cout << endl;
        break;
    case 3:
        cout << "Preorder Traversal: ";
        bst.preorderTraversal(bst.root);
        cout << endl;
        break;
    case 4:
        cout << "Postorder Traversal: ";
        bst.postorderTraversal(bst.root);
        cout << endl;
        break;
    case 5:
        cout << "Enter the element to search: ";
        cin >> key;
        if (bst.searchElement(bst.root, key)) {
            cout << "Element " << key << " found in the BST." << endl;
        } else {
            cout << "Element " << key << " not found in the BST." << endl;
        }
        break;
    case 6:
        cout << "Exiting the program..." << endl;
        break;
    default:
        cout << "Invalid choice. Please try again." << endl;
        break;
}
cout << endl;
```

```
    } while (choice != 6);
    return 0;
}
```

Output-

```
Binary Search Tree (BST) Menu
1. Create a BST of N Integers
2. Traverse the BST in Inorder
3. Traverse the BST in Preorder
4. Traverse the BST in Postorder
5. Search the BST for a given element
6. Exit
Enter your choice: 1
BST created with elements: 6 9 5 2 8 15 24 14 7 8 5 2

Binary Search Tree (BST) Menu
1. Create a BST of N Integers
2. Traverse the BST in Inorder
3. Traverse the BST in Preorder
4. Traverse the BST in Postorder
5. Search the BST for a given element
6. Exit
Enter your choice: 2
Inorder Traversal: 2 2 5 5 6 7 8 8 9 14 15 24
```

```
Binary Search Tree (BST) Menu
1. Create a BST of N Integers
2. Traverse the BST in Inorder
3. Traverse the BST in Preorder
4. Traverse the BST in Postorder
5. Search the BST for a given element
6. Exit
Enter your choice: 3
Preorder Traversal: 6 5 2 2 5 9 8 7 8 15 14 24

Binary Search Tree (BST) Menu
1. Create a BST of N Integers
2. Traverse the BST in Inorder
3. Traverse the BST in Preorder
4. Traverse the BST in Postorder
5. Search the BST for a given element
6. Exit
Enter your choice: 4
Postorder Traversal: 2 5 2 5 8 7 8 14 24 15 9 6
```

12. Implement linear search and binary with analysis of time and space complexity.

Answer:

A-

```
#include <iostream>
using namespace std;
int main()
{
int arr[10], i, num, index;
cout << "Enter 10 Numbers: ";
for (i = 0; i < 10; i++)
cin >> arr[i];
cout << "\nEnter a Number to Search: ";
cin >> num;
for (i = 0; i < 10; i++)
{
if (arr[i] == num)
{
index = i;
break;
}
}
cout << "\nFound at Index No." << index;
cout << endl;
return 0;
}
```

Output-

```
Enter 10 Numbers: 8
3
6
5
4
1
9
7
8
12

Enter a Number to Search: 1

Found at Index No.5
```

B-

```
#include <iostream>
using namespace std;

int main() {
    int i, arr[10], num, first, last, middle;
    cout << "Enter 10 Elements (in ascending order): ";
    for (i = 0; i < 10; i++)
        cin >> arr[i];
    cout << "\nEnter Element to be Searched: ";
    cin >> num;
    first = 0;
    last = 9;
    middle = (first + last) / 2;
    while (first <= last) {
        if (arr[middle] < num)
            first = middle + 1;
        else if (arr[middle] == num) {
            cout << "\nThe number, " << num << ", found at Position " << middle + 1;
            break;
        } else
            last = middle - 1;
        middle = (first + last) / 2;
    }
    if (first > last)
        cout << "\nThe number, " << num << ", is not found in the given Array";
    cout << endl;
    return 0;
}
```

Output-

```
Enter 10 Elements (in ascending order): 2
6
7
9
12
14
16
18
20
22

Enter Element to be Search: 18

The number, 18 found at Position 8
```